



WEEK 7 ASSIGNMENT

Spring 2026

Pol Monné Parera (L30170743)
pmonneparera@lewisu.edu

1. Technical Description

1.1 Image Source

For this assignment I chose to develop the digital image segmentation task. The experiment uses three bundled Matplotlib sample images, so the notebook is reproducible without requiring external downloads.

Image	Source
Grace Hopper portrait	Matplotlib sample data: grace_hopper.jpg
Matplotlib logo	Matplotlib sample data: logo2.png
Present icon	Matplotlib sample data: Minduka_Present_Blue_Pack.png

1.2 CNN Segmentation Technique

The network utilized consists of a small convolutional encoder/decoder. The encoder uses convolution and max-pooling layers to learn local image patterns while reducing spatial size, while the decoder uses bilinear up sampling and convolution layers to recover a full-resolution mask.

Since the data did not contain manually annotated masks, the target labels were created with with an Otsu-style thresholding rule and then used as supervised training labels. This approach follows the same logic as the provided coding example, where synthetic masks are manually created based on intensity before training the CNN.

2. Design of the Algorithm

2.1 Workflow Summary

The workflow followed to implement the assignment is defined below as the defined consecutive steps:

- Load three sample images and resize each image to 160 x 160 pixels.
- Convert each image to grayscale and normalize pixel values to the range [0, 1].
- Generate a binary target mask for each image using threshold-based segmentation.
- Create augmented training samples with flips, rotations, shifts, and light noise.
- Train the CNN with binary cross-entropy loss.
- Run inference on the original images and compare predicted masks against target masks.
- Measure pixel accuracy, Intersection over Union (IoU), and Dice score.

2.2 Pseudocode

LOAD three source images

FOR each image:

 RESIZE image and CONVERT to grayscale

 CREATE binary threshold mask

 AUGMENT image/mask pairs

DEFINE encoder-decoder CNN

TRAIN CNN using BCEWithLogitsLoss

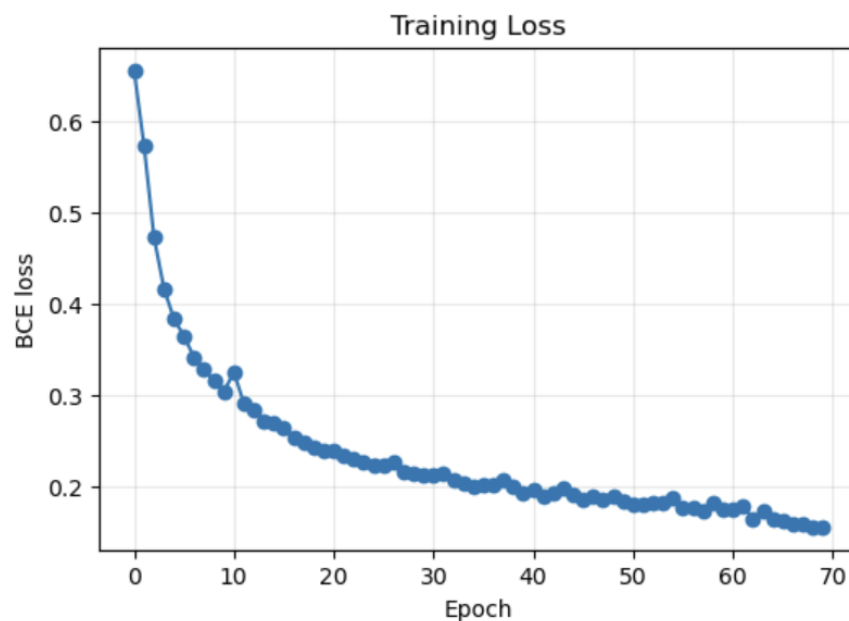
PREDICT segmentation mask for each original image

COMPUTE accuracy, IoU, and Dice score

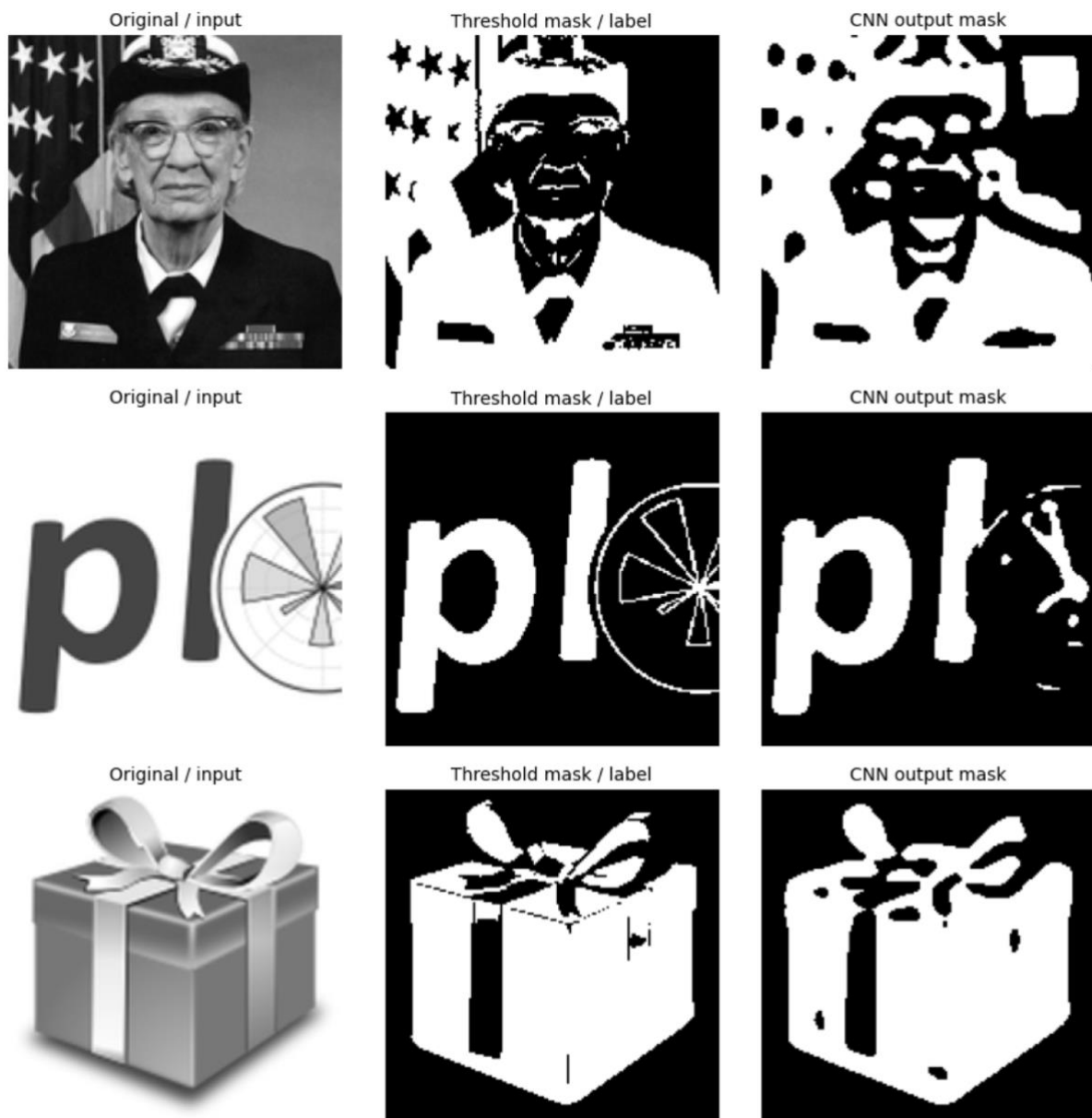
SAVE before/after visual comparisons

3. Results of the Algorithm

3.1 Training Loss Curve of the CNN



3.2 Image Segmentation Comparison



3.3 Quantitative Summary

	Image	Pixel Accuracy	IoU	Dice
0	Grace Hopper portrait	0.8717	0.8025	0.8904
1	Matplotlib logo	0.9541	0.8121	0.8963
2	Present icon	0.9613	0.9218	0.9593

4. Analysis of the Results

4.1 Did I obtain what I expected?

The results obtained in the assignment matched the initial expectations for a small CNN trained on synthetic masks. The model clearly learned the dominant foreground and background structure in all examples producing masks similar to the threshold labels, with the best results occurring where object boundary is simple and the contrast is higher (e.g. the present icon). The portrait image was interesting to analyze as it proved to be harder because the threshold label contains many fine details, generating a smoother CNN output.

4.2 Observations and Limitations

The most relevant limitation of the study comes from the automatically generated labels, as in real life the labels are normally drawn by a human annotator. As a result, the quantitative metrics evaluate how well the CNN learned the threshold rule, not as much if the segmentation is semantically perfect or not. To obtain a stronger production-style segmentation model, a larger real dataset with manually labeled masks would be required.

4.3 Conclusions

The experiment studied in this project is able to demonstrate the complete segmentation workflow requested in the assignment, involving image processing, CNN model design, training, inference, visualization, and analysis of the results.

The project is small, reproducible and aligned with the requirements. Nevertheless, some future steps that could improve the project general structure and results could include:

- Use manually annotated masks instead of threshold-generated synthetic masks on a larger dataset.
- Add validation data to monitor overfitting during training.
- Test the model on completely unseen images.
- Apply post-processing to clean noisy mask edges.

5. Program Listings

The complete coding implementation is provided in the companion python script “assignment7_cnn_image_segmentation.py”. The code has been downloaded from the original Jupyter Notebook file hosted on Google Colab with a python extension in order to comply with the statement requirements. Furthermore, the more relevant code snippets are also displayed below:

```
def make_mask(gray, strategy):
    if strategy == "dark_foreground":
        return (gray < otsu_threshold(gray)).astype(np.float32)
    if strategy == "mid_high":
        return (gray > np.quantile(gray, 0.58)).astype(np.float32)
    raise ValueError(strategy)

for item in images:
    item["gray"] = rgb_to_gray(item["rgb"])
    item["mask"] = make_mask(item["gray"], item["strategy"])

fig, axes = plt.subplots(3, 2, figsize=(7, 9))
for row, item in enumerate(images):
    axes[row, 0].imshow(item["gray"], cmap="gray")
    axes[row, 0].set_title("Input image")
    axes[row, 1].imshow(item["mask"], cmap="gray")
    axes[row, 1].set_title("Training mask")
    for ax in axes[row]:
        ax.axis("off")
plt.tight_layout()
plt.show()
```

Figure 1 Synthetic Mask Generation

```
class SmallSegmentationCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False),
            nn.Conv2d(32, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False),
            nn.Conv2d(16, 1, kernel_size=1),
        )

    def forward(self, x):
        return self.network(x)

model = SmallSegmentationCNN()
print(model)
```

Figure 2 CNN Model

```

criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=5e-4)
loss_history = []

for epoch in range(1, 71):
    epoch_loss = 0.0
    model.train()
    for batch_x, batch_y in loader:
        optimizer.zero_grad()
        logits = model(batch_x)
        loss = criterion(logits, batch_y)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * batch_x.size(0)
    epoch_loss /= len(loader.dataset)
    loss_history.append(epoch_loss)
    if epoch == 1 or epoch % 5 == 0:
        print(f"Epoch {epoch:02d}: loss={epoch_loss:.4f}")

plt.figure(figsize=(6, 4))
plt.plot(loss_history, marker="o")
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("BCE loss")
plt.grid(alpha=0.3)
plt.show()

```

Figure 3 Model Training

Resources

Matplotlib Development Team. Matplotlib sample data. GitHub repository.
https://github.com/matplotlib/matplotlib/tree/main/lib/matplotlib/mpl-data/sample_data